LAB 10: Software Metrics & Complexity Analysis

Name: Gyan Prakash Tiwari
Roll No.: 202401100200146
Branch: CSE

Project: AI Chatbot for University Helpdesk (RAG-based) |
github.com/Xxierrr/AI-Chatbot-for-University-Helpdesk

1. Aim

To apply software metrics – size, cyclomatic complexity, maintainability, coupling and cohesion – to the University Helpdesk Chatbot and interpret the results.

2. Metrics Used

Metric	Definition / Formula	Why it matters here
LOC / SLOC	Count of lines (source lines excluding comments/blank).	Estimates size and effort of each module.
Cyclomatic Complexity V(G)	V(G) = E – N + 2P = number of decision points + 1	Number of independent paths = minimum test cases.
Halstead Volume	$V = N \times \log_2(n)$, with N = total operators+operands and n = distinct ones.	Measures information content of code.
Maintainability Index	MI = max(0, (171 – 5.2 ln V – 0.23 G – 16.2 ln LOC) × 100/171)	Single score of how easy code is to maintain.
Fan-in / Fan-out	Ca = modules depending on this one; Ce = modules this one depends on.	Shows coupling hot spots.
Instability	I = Ce / (Ca + Ce)	0 = stable, 1 = unstable.
LCOM / Cohesion	Lack of cohesion of methods within a class/module.	Low LCOM means focused modules.

3. Worked Example: Cyclomatic Complexity of the Chat Handler

Consider the chat request handler (POST /api/chat). Its decisions are: (1) is the token valid, (2) is the message non-empty, (3) does the conversation already exist, (4) were relevant chunks retrieved, (5) did the LLM call succeed.

```
def handle_chat(req, user):
  if not valid_token(user): raise 401    # D1
  if not req.message.strip(): raise 422    # D2
  conv = get(req.conv_id) or create()    # D3
  chunks = retrieve(req.message)
  if not chunks: log_unanswered(); return fallback  # D4
  try: answer = llm(prompt)    # D5
  except Error: answer = error_fallback
  save(conv, answer); return answer
```

Control-flow graph count: Nodes N = 13, Edges E = 17, connected components P = 1.

V(G) = E – N + 2P = 17 – 13 + 2 = 6. Check by decision points: 5 predicates + 1 = **6**. So at least 6 test cases are required (invalid token, empty message, new conversation, existing conversation with no

chunks, success path, and LLM failure).

4. Complexity Risk Scale

V(G)	Risk	Action
1 – 10	Simple, low risk	Acceptable
11 – 20	Moderate	Consider refactoring
21 – 50	Complex, high risk	Refactor
> 50	Untestable	Redesign

The chat handler (V(G) = 6) is in the simple, low-risk band.

5. Maintainability Index Example

For the handler above, assume Halstead volume V = 800, V(G) = 6, LOC = 45. Then MI = 171 − 5.2 ln(800) − 0.23(6) − 16.2 ln(45) = 73.2; normalised to a 0–100 scale this gives **MI = 42.8**. On the Visual Studio scale (20–100 = good maintainability) the function is easy to maintain.

6. Module-Level Metrics (Illustrative Estimates)

Note: The values below are design-level estimates for the planned modules, made to demonstrate the method. For final submission replace them with the output of the tools listed in Section 7 run on the actual source code.

Module	Est. SLOC	Est. V(G)	Ca	Ce	I = Ce/(Ca+Ce)	Comment
main.py	60	3	0	4	1.00	Entry point; composes routers.
routes/chat	90	6	1	3	0.75	Request validation and orchestration.
auth_service.py	110	8	3	2	0.40	Widely used, fairly stable.
rag_service.py	150	9	2	4	0.67	Highest coupling; split retriever/LLM later.
ingest_documents.py	120	10	1	3	0.75	Branches per file type.
React components	~800	< 10 each	–	–	–	Keep components small and reusable.

7. How to Measure on the Real Code

```
# Python back end
pip install radon lizard
radon raw backend/        # LOC, SLOC, comments
radon cc -s -a backend/   # cyclomatic complexity per function
radon mi -s backend/    # maintainability index
lizard backend/ frontend/src  # CCN for Python and JS/JSX

# React front end
npx eslint src --rule "complexity: [warn, 10]"
```

8. Interpretation and Recommendations

- Keep every function at V(G) ≤ 10; split *ingest_documents.py* into one loader per file type to reduce branching.

- *rag_service.py* has the highest coupling, so extract retriever and LLM-client classes behind interfaces.
- Use V(G) as a test-planning aid: each independent path needs at least one pytest case.
- Track MI and complexity in CI to prevent quality from degrading as features (multilingual, voice) are added.

9. Conclusion

Metric analysis shows the chatbot is built from small, low-complexity functions with coupling concentrated in the RAG service. Applying the measurements regularly will keep the code maintainable and make test effort predictable.